Human Verification: `Sequential`

The Sequential Operator $\triangleleft$ and its Monoidal Structure

Machine-generated from `lean/Sequential.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Sequential` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean.

The module is a near-duplicate of `Composition.lean`, re-exposing the same monoidal structure under the name "sequential operator" rather than "composition product". The mathematical content is identical; only the terminology and the names of a few definitions differ (`Container.seq` vs `Container.comp`, `seqExt` vs `compExt`). A fidelity note at the end records the relationship.

Contents

1 Overview **1**

2 The sequential operator **2**
2.1 Definition: `Container.seq` . . . 2
2.2 Definition: `Container.I` . . . 2

3 Extension isomorphism **3**
3.1 Definitions: `Container.seqExt` and `Container.seqExtInv` . . . 3
3.2 Theorems: `seqExtInv_seqExt` and `seqExt_seqExtInv` . . . 3

4 Container isomorphisms **3**
4.1 Structure: `ContainerIso` . . . 3

5 Monoidal coherences **4**
5.1 Definition: `Container.leftUnitor` . . . 4
5.2 Definition: `Container.rightUnitor` . . . 4
5.3 Definition: `Container.associator` . . . 5

6 The compositional payoff **5**

7 Relationship to `Composition.lean` **5**

8 Translation conventions **5**

1 Overview

The sequential operator $\triangleleft$ on containers has the same formula as the composition product of `Composition.lean`: for $F = S \triangleleft P$ and $G = T \triangleleft Q$,

- shapes of $G \triangleleft F$: $\Sigma_{t:T}(Q(t) \to S)$, and
- positions at (t, f): $\Sigma_{q:Q(t)} P(f(q))$.

The extension satisfies $[\![G \triangleleft F]\!] = [\![G]\!] \circ [\![F]\!]$. The monoidal unit is the same identity container I. The coherence isomorphisms (left/right unitors, associator) have identical statements and proofs.

The module's docstring frames the same structure as the "sequential operator" to emphasise the operational reading: $G \triangleleft F$ is "run F, then run G on the result", and a $\triangleleft$-comonoid is a directed container.

2 The sequential operator

2.1 Definition: `Container.seq`

Definition 2.1 (Sequential operator)**.** For containers G and F, their **sequential composition** $G \triangleleft F$ ("G outside, F inside") is:

$$\mathrm{Shape}(G \triangleleft F) = \Sigma_{t:G.\mathrm{Shape}} (G.\mathrm{Pos}(t) \to F.\mathrm{Shape}),$$

$$\mathrm{Pos}(G \triangleleft F)(t, f) = \Sigma_{q:G.\mathrm{Pos}(t)} F.\mathrm{Pos}(f(q)).$$

The extension satisfies $[\![G \triangleleft F]\!]X \cong [\![G]\!]([\![F]\!]X)$.

Lean.

```
def Container.seq (G F : Container) : Container where
  Shape := (t : G.Shape) * (G.Pos t → F.Shape)
  Pos := fun tf ⇒ (q : G.Pos tf.1) * F.Pos (tf.2 q)

scoped infixr:70 "␣seq␣" ⇒ Container.seq
-- (written with the unicode diamond operator in Lean source)
```

Haskell.

```
-- G `seq` F: shapes = (t, p_g t → s_f), positions = (p_g t, p_f (f q
   ))
-- Same as Container.comp; see fidelity note.
data Seq g p_g f p_f x = Seq g (p_g → f) (p_g → p_f → x)
```

Fidelity note. `Container.seq` and `Container.comp` (from `Composition.lean`) are definitionally identical — same formula, same type. The duplication appears to be a development artifact, with `Sequential.lean` providing an alternative narrative framing of the same monoidal structure. The verifier should treat the two files as presenting the same mathematics.

2.2 Definition: `Container.I`

Definition 2.2 (Identity container)**.** The identity container I has one shape and one position; its extension is the identity functor.

Lean.

```
def Container.I : Container where
  Shape := Unit
  Pos := fun _ ⇒ Unit
```

Haskell.

```
newtype Id x = Id x    -- one shape, one position
```

3 Extension isomorphism

3.1 Definitions: `Container.seqExt` and `Container.seqExtInv`

Definition 3.1 (Extension isomorphism for the sequential operator)**.** There is a natural bijection

$$\mathrm{seqExt} : [\![G \triangleleft F]\!]X \xrightarrow{\sim} [\![G]\!]([\![F]\!]X),$$

$$\mathrm{seqExtInv} : [\![G]\!]([\![F]\!]X) \xrightarrow{\sim} [\![G \triangleleft F]\!]X.$$

The formulas are identical to those of `compExt` / `compExtInv`.

Lean.

```
def Container.seqExt {G F : Container} {X : Type} :
    Ext (G seq F) X → Ext G (Ext F X) :=
  fun p ⇒ ⟨p.1.1, fun q ⇒ ⟨p.1.2 q, fun r ⇒ p.2 ⟨q, r⟩⟩⟩

def Container.seqExtInv {G F : Container} {X : Type} :
    Ext G (Ext F X) → Ext (G seq F) X :=
  fun p ⇒ ⟨⟨p.1, fun q ⇒ (p.2 q).1⟩, fun qr ⇒ (p.2 qr.1).2 qr.2⟩
```

Haskell.

```
seqExt :: Seq g p_g f p_f x → (g, p_g → (f, p_f → x))
seqExt (Seq t assign lab) = (t, \q → (assign q, \r → lab q r))

seqExtInv :: (g, p_g → (f, p_f → x)) → Seq g p_g f p_f x
seqExtInv (t, m) = Seq t (fst . m) (\q r → snd (m q) r)
```

3.2 Theorems: `seqExtInv_seqExt` and `seqExt_seqExtInv`

Theorem 3.2 (Extension isomorphism is an isomorphism)**.**

$$\mathrm{seqExtInv} \circ \mathrm{seqExt} = \mathrm{id}, \qquad \mathrm{seqExt} \circ \mathrm{seqExtInv} = \mathrm{id}.$$

Both hold definitionally (by `rfl` after `cases`).

Lean.

```
theorem Container.seqExtInv_seqExt {G F : Container} {X : Type}
    (p : Ext (G seq F) X) :
    Container.seqExtInv (Container.seqExt p) = p

theorem Container.seqExt_seqExtInv {G F : Container} {X : Type}
    (p : Ext G (Ext F X)) :
    Container.seqExt (Container.seqExtInv p) = p
```

Haskell.

```
-- seqExtInv . seqExt = id   (and vice versa)
-- Both hold by eta-laws for pairs.
```

4 Container isomorphisms

4.1 Structure: `ContainerIso`

Definition 4.1 (Container isomorphism)**.** An isomorphism in **Cont**: a pair of mutually inverse container morphisms. (Identical definition to `Composition.lean`.)

Lean.

```
structure ContainerIso (C D : Container) where
  hom : ContainerMorphism C D
  inv : ContainerMorphism D C
  hom_inv : hom.comp inv = ContainerMorphism.id C
  inv_hom : inv.comp hom = ContainerMorphism.id D
```

Haskell.

```
data ContainerIso c d = ContainerIso
  { hom :: Morph c d
  , inv :: Morph d c
  -- hom_inv and inv_hom are equational laws
  }
```

5 Monoidal coherences

5.1 Definition: `Container.leftUnitor`

Definition 5.1 (Left unitor). $\lambda_F : I \triangleleft F \cong F$. A shape $(\langle\rangle, f)$ of $I \triangleleft F$ bijects with F.Shape via $f \mapsto f(\langle\rangle)$.

Lean.

```
def Container.leftUnitor (F : Container) :
    ContainerIso (Container.I seq F) F where
  hom :=
    { onShapes := fun tf ⇒ tf.2 ()
      onPos := fun _ p ⇒ ⟨(), p⟩ }
  inv :=
    { onShapes := fun s ⇒ ⟨(), fun _ ⇒ s⟩
      onPos := fun _ qp ⇒ qp.2 }
  hom_inv := rfl
  inv_hom := rfl
```

Haskell.

```
leftUnitor :: ContainerIso (I `Seq` F) F
-- Same as in Composition.lean.
```

5.2 Definition: `Container.rightUnitor`

Definition 5.2 (Right unitor). $\rho_F : F \triangleleft I \cong F$. Shapes project onto F.Shape; positions collapse $\Sigma_{p:F.\mathrm{Pos}(s)}\mathrm{Unit} \cong F.\mathrm{Pos}(s)$.

Lean.

```
def Container.rightUnitor (F : Container) :
    ContainerIso (F seq Container.I) F where
  hom :=
    { onShapes := fun sg ⇒ sg.1
      onPos := fun _ p ⇒ ⟨p, ()⟩ }
  inv :=
    { onShapes := fun s ⇒ ⟨s, fun _ ⇒ ()⟩
      onPos := fun _ pu ⇒ pu.1 }
  hom_inv := rfl
  inv_hom := rfl
```

Haskell.

```
rightUnitor :: ContainerIso (F `Seq` I) F
-- Same as in Composition.lean.
```

5.3 Definition: `Container.associator`

Definition 5.3 (Associator). $\alpha_{H,G,F} : (H \triangleleft G) \triangleleft F \cong H \triangleleft (G \triangleleft F)$. On shapes: curries the inner assignment. On positions: re-associates the nested dependent sum $((u, v), p) \leftrightarrow (u, (v, p))$. No transport needed (position fibres agree definitionally).

Lean.

```
def Container.associator (H G F : Container) :
    ContainerIso ((H seq G) seq F) (H seq (G seq F)) where
  hom :=
    { onShapes := fun xk ⇒
        ⟨xk.1.1, fun u ⇒ ⟨xk.1.2 u, fun v ⇒ xk.2 ⟨u, v⟩⟩⟩
      onPos := fun _ uvp ⇒ ⟨⟨uvp.1, uvp.2.1⟩, uvp.2.2⟩ }
  inv :=
    { onShapes := fun am ⇒
        ⟨⟨am.1, fun u ⇒ (am.2 u).1⟩, fun y ⇒ (am.2 y.1).2 y.2⟩
      onPos := fun _ yp ⇒ ⟨yp.1.1, yp.1.2, yp.2⟩ }
  hom_inv := ...   -- ContainerMorphism.ext' rfl (cases + rfl)
  inv_hom := ...   -- ContainerMorphism.ext' rfl (cases + rfl)
```

Haskell.

```
associator :: ContainerIso ((H `Seq` G) `Seq` F) (H `Seq` (G `Seq` F))
-- Same formulas as in Composition.lean.
```

6 The compositional payoff

A **$\triangleleft$-comonoid** in $(\mathbf{Cont}, \triangleleft, I)$ is a container C with a counit $\varepsilon : C \to I$ and a comultiplication $\delta : C \to C \triangleleft C$, satisfying the comonoid laws. This is exactly a **directed container** (a comonad on **Set** whose functor is a container). The sequential framing makes the operational reading explicit: a directed container is a container that can be "stepped", with the comonad structure recording how steps compose.

7 Relationship to `Composition.lean`

Fidelity note. `Sequential.lean` and `Composition.lean` define mathematically identical structures: `Container.seq` = `Container.comp`, `seqExt` = `compExt`, and so on. The files differ only in terminology and narrative framing:

- `Composition.lean` emphasises $[\![G \triangleleft F]\!] = [\![G]\!] \circ [\![F]\!]$ as functor composition;
- `Sequential.lean` emphasises the sequential reading and the comonoid/directed-container connection.

In a production codebase these would be merged into one file. As written, they coexist by declaring the notation in separate scopes; the notation `infixr:70 " ◁ "` is declared twice, once per file.

8 Translation conventions

1. **Proof bodies omitted.** Lean machine-checks all proofs.
2. **Definitional round trips.** All four round-trip equalities for `seqExt`/`seqExtInv` and the two unitor/associator laws hold by `rfl` after `cases`, using only definitional reduction.
3. **Unicode notation.** The Lean source uses the diamond operator $\triangleleft$ (`triangleleft`). The listings use `seq` as an ASCII placeholder.

4. **Dependent type limitation.** Haskell approximations drop the dependency of position type on shape value.